FE515 Lecture 7 Walkthrough: Root Finding and Optimization

Contents

1 Goal for Today **2**

2 Root Finding Motivation **2**

3 Bisection Method **3**

3.1 Main Idea . . . 3

3.2 Manual First Iteration . . . 3

3.3 A Basic Bisection Loop . . . 4

3.4 Writing Bisection as a Function . . . 4

3.5 Tracking the Iterations . . . 5

4 Newton-Raphson Method **8**

4.1 Main Idea . . . 8

4.2 Implement Newton-Raphson in R . . . 8

4.3 Tracking Newton-Raphson Iterations . . . 9

5 Financial Example: Solving for a Bond Yield **10**

5.1 Compare to Bisection and `uniroot` . . . 11

6 Gradient Descent **12**

6.1 Simple Quadratic Example . . . 12

6.2 Step Size Matters . . . 13

7 Local Minima Example **15**

8 Newton's Method for Optimization **17**

9 Wrap-Up **19**

1 Goal for Today

This walkthrough introduces two related computational problems:

1. **Root finding:** solve equations of the form

$$f(x) = 0.$$

2. **Optimization:** find the value of x that minimizes a function

$$\min_x f(x).$$

The goal is to understand how the algorithms work, implement them, and then compare our implementations to R's built-in tools.

2 Root Finding Motivation

Suppose we want to solve

$$x^2 - 2 = 0.$$

We know the exact answers are

$$x = \pm\sqrt{2}.$$

But in many financial and statistical problems, we cannot solve the equation exactly. Instead, we define a function and ask the computer to find the input where the function equals zero.

```
f <- function(x) {
  x^2 - 2
}

f(-2)
```

```
## [1] 2
```

```
f(-1)
```

```
## [1] -1
```

Since $f(-2) > 0$ and $f(-1) < 0$, the function crosses zero somewhere between -2 and -1. This is the setup for the bisection method.

3 Bisection Method

3.1 Main Idea

The bisection method is based on the Intermediate Value Theorem.

If f is continuous on $[a, b]$, and $f(a)$ and $f(b)$ have opposite signs, then there must be at least one root in $[a, b]$.

The algorithm repeatedly cuts the interval in half:

$$c = \frac{a + b}{2}.$$

Then we check whether the root is in the left half $[a, c]$ or the right half $[c, b]$.

3.2 Manual First Iteration

Start with

$$a = -2, \qquad b = -1.$$

```
a <- -2
b <- -1

c <- (a + b) / 2

f(a)
```

```
## [1] 2
```

```
f(b)
```

```
## [1] -1
```

```
c
```

```
## [1] -1.5
```

```
f(c)
```

```
## [1] 0.25
```

Since $f(a)$ and $f(c)$ have the same sign, the root is not in $[a, c]$. The root must be in $[c, b]$, so we replace a with c.

```
a <- c
a
```

```
## [1] -1.5
```

```r
b
```

```
## [1] -1
```

3.3 A Basic Bisection Loop

Now we automate that logic.

```r
a <- -2
b <- -1
tol <- 0.001

while (TRUE) {
  c <- (a + b) / 2

  if (f(c) == 0 || abs(b - a) < tol) {
    break
  }

  if (f(a) * f(c) < 0) {
    b <- c
  } else {
    a <- c
  }
}

c
```

```
## [1] -1.414551
```

```r
f(c)
```

```
## [1] 0.0009539127
```

The value of `c` is our numerical approximation to the root.

3.4 Writing Bisection as a Function

A reusable function should take the function, interval, tolerance, and maximum number of iterations as inputs.

```r
bisection <- function(f, a, b, tol = 0.001, N.max = 100) {

  f.a <- f(a)
  f.b <- f(b)

  if (is.na(f.a) || is.na(f.b)) {
    return(NA)
  }
```

```
  if (f.a == 0) {
    return(a)
  }

  if (f.b == 0) {
    return(b)
  }

  if (f.a * f.b > 0) {
    warning("f(a) and f(b) have the same sign. Bisection may not find a valid root.")
  }

  for (n in 1:N.max) {
    c <- (a + b) / 2
    f.c <- f(c)

    if (f.c == 0 || abs(b - a) < tol) {
      return(c)
    }

    if (f.a * f.c < 0) {
      b <- c
      f.b <- f.c
    } else {
      a <- c
      f.a <- f.c
    }
  }

  warning("Maximum number of iterations reached.")
  return(c)
}

bisection(f, -2, -1)
```

```
## [1] -1.414551
```

```
bisection(f, -1, -2)
```

```
## [1] -1.414551
```

3.5 Tracking the Iterations

It is often easier to learn an algorithm if we can see the path it takes.

```
bisection_history <- function(f, a, b, tol = 0.001, N.max = 100) {

  f.a <- f(a)
  f.b <- f(b)

  if (is.na(f.a) || is.na(f.b) || f.a * f.b > 0) {
```

```
    stop("Bisection requires f(a) and f(b) to have opposite signs.")
  }

  history <- data.frame(
    iteration = integer(),
    a = numeric(),
    b = numeric(),
    c = numeric(),
    f.c = numeric(),
    interval_width = numeric()
  )

  for (n in 1:N.max) {
    c <- (a + b) / 2
    f.c <- f(c)

    history <- rbind(
      history,
      data.frame(
        iteration = n,
        a = a,
        b = b,
        c = c,
        f.c = f.c,
        interval_width = abs(b - a)
      )
    )

    if (f.c == 0 || abs(b - a) < tol) {
      break
    }

    if (f.a * f.c < 0) {
      b <- c
      f.b <- f.c
    } else {
      a <- c
      f.a <- f.c
    }
  }

  return(history)
}

bis_hist <- bisection_history(f, -2, -1)

head(bis_hist, 10)
```

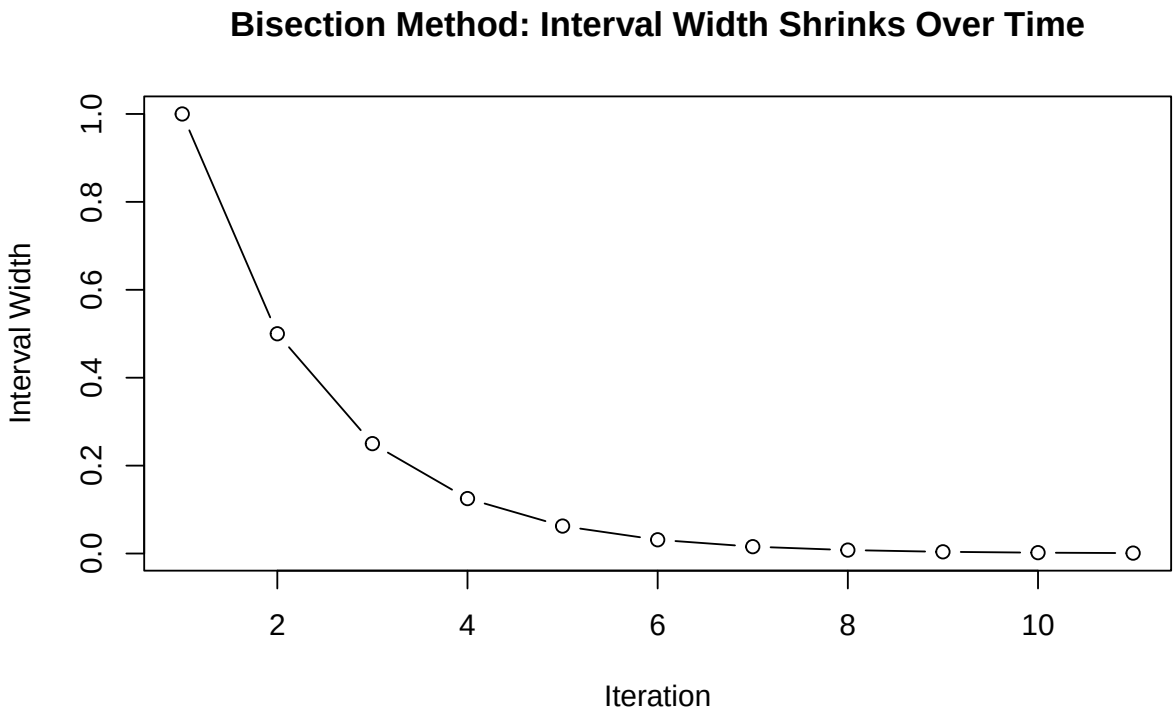
```
##    iteration         a         b         c           f.c interval_width
## 1          1 -2.000000 -1.000000 -1.500000  0.2500000000    1.000000000
## 2          2 -1.500000 -1.000000 -1.250000 -0.4375000000    0.500000000
## 3          3 -1.500000 -1.250000 -1.375000 -0.1093750000    0.250000000
## 4          4 -1.500000 -1.375000 -1.437500  0.0664062500    0.125000000
```

```
## 5          5 -1.437500 -1.375000 -1.406250 -0.0224609375    0.062500000
## 6          6 -1.437500 -1.406250 -1.421875  0.0217285156    0.031250000
## 7          7 -1.421875 -1.406250 -1.414062 -0.0004272461    0.015625000
## 8          8 -1.421875 -1.414062 -1.417969  0.0106353760    0.007812500
## 9          9 -1.417969 -1.414062 -1.416016  0.0051002502    0.003906250
## 10        10 -1.416016 -1.414062 -1.415039  0.0023355484    0.001953125
```

```r
tail(bis_hist, 5)
```

```
##    iteration         a         b         c           f.c interval_width
## 7          7 -1.421875 -1.406250 -1.414062 -0.0004272461   0.0156250000
## 8          8 -1.421875 -1.414062 -1.417969  0.0106353760   0.0078125000
## 9          9 -1.417969 -1.414062 -1.416016  0.0051002502   0.0039062500
## 10        10 -1.416016 -1.414062 -1.415039  0.0023355484   0.0019531250
## 11        11 -1.415039 -1.414062 -1.414551  0.0009539127   0.0009765625
```

```r
plot(
  bis_hist$iteration,
  bis_hist$interval_width,
  type = "b",
  xlab = "Iteration",
  ylab = "Interval Width",
  main = "Bisection Method: Interval Width Shrinks Over Time"
)
```



4 Newton-Raphson Method

4.1 Main Idea

Newton-Raphson uses the tangent line to approximate the function near the current point.

Starting from x_0, we update according to

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Then we repeat the process:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Unlike bisection, Newton-Raphson requires a derivative.

4.2 Implement Newton-Raphson in R

For the same example,

$$f(x) = x^2 - 2,$$

the derivative is

$$f'(x) = 2x.$$

```
Newton_Raphson <- function(f, df, x0, tol = 0.001, N.max = 100) {

  for (n in 1:N.max) {

    derivative <- df(x0)

    if (derivative == 0) {
      stop("Derivative is zero. Newton-Raphson cannot continue.")
    }

    x1 <- x0 - f(x0) / derivative

    if (abs(x1 - x0) < tol) {
      return(x1)
    }

    x0 <- x1
  }

  warning("Maximum number of iterations reached.")
  return(x1)
}

df <- function(x) {
```

```r
  2 * x
}

Newton_Raphson(f, df, -1)
```

```
## [1] -1.414214
```

4.3 Tracking Newton-Raphson Iterations

```r
Newton_Raphson_history <- function(f, df, x0, tol = 0.001, N.max = 100) {

  history <- data.frame(
    iteration = integer(),
    x0 = numeric(),
    f.x0 = numeric(),
    df.x0 = numeric(),
    x1 = numeric(),
    step_size = numeric()
  )

  for (n in 1:N.max) {

    derivative <- df(x0)

    if (derivative == 0) {
      stop("Derivative is zero. Newton-Raphson cannot continue.")
    }

    x1 <- x0 - f(x0) / derivative

    history <- rbind(
      history,
      data.frame(
        iteration = n,
        x0 = x0,
        f.x0 = f(x0),
        df.x0 = derivative,
        x1 = x1,
        step_size = abs(x1 - x0)
      )
    )

    if (abs(x1 - x0) < tol) {
      break
    }

    x0 <- x1
  }

  return(history)
}
```

```r
newton_hist <- Newton_Raphson_history(f, df, -1)

newton_hist
```

```
##   iteration        x0          f.x0     df.x0        x1    step_size
## 1         1 -1.000000 -1.000000e+00 -2.000000 -1.500000 0.5000000000
## 2         2 -1.500000  2.500000e-01 -3.000000 -1.416667 0.0833333333
## 3         3 -1.416667  6.944444e-03 -2.833333 -1.414216 0.0024509804
## 4         4 -1.414216  6.007305e-06 -2.828431 -1.414214 0.0000021239
```

The table below also tracks the number of iterations. This lets us compare not only the final answer, but also how quickly each method gets there under the chosen stopping rule.

```r
bisection_result <- bisection(f, -2, -1)
newton_result <- Newton_Raphson(f, df, -1)

bisection_iterations <- nrow(bisection_history(f, -2, -1))
newton_iterations <- nrow(Newton_Raphson_history(f, df, -1))

data.frame(
  method = c("Bisection", "Newton-Raphson"),
  approximate_root = c(
    bisection_result,
    newton_result
  ),
  iterations = c(
    bisection_iterations,
    newton_iterations
  )
)
```

```
##           method approximate_root iterations
## 1      Bisection        -1.414551         11
## 2 Newton-Raphson        -1.414214          4
```

5 Financial Example: Solving for a Bond Yield

Suppose a 2-year bond has principal \$100 and pays a 6% annual coupon semiannually. That means it pays \$3 every six months, and \$103 at maturity.

If the bond price is \$98.39, the continuously compounded yield y solves

$$3e^{-0.5y} + 3e^{-y} + 3e^{-1.5y} + 103e^{-2y} = 98.39.$$

Move everything to one side:

$$f(y) = 3e^{-0.5y} + 3e^{-y} + 3e^{-1.5y} + 103e^{-2y} - 98.39.$$

Then the yield is the root of $f(y)$.

```r
bond_price_error <- function(y) {
  3 * exp(-0.5 * y) +
    3 * exp(-1 * y) +
    3 * exp(-1.5 * y) +
    103 * exp(-2 * y) -
    98.39
}

bond_price_error_derivative <- function(y) {
  -1.5 * exp(-0.5 * y) -
    3 * exp(-1 * y) -
    4.5 * exp(-1.5 * y) -
    206 * exp(-2 * y)
}

Newton_Raphson(
  f = bond_price_error,
  df = bond_price_error_derivative,
  x0 = 0.02
)
```

```
## [1] 0.06759816
```

The output is approximately `0.0676`, so the continuously compounded yield is about 6.76%.

5.1 Compare to Bisection and `uniroot`

R has a built-in root finding function called `uniroot()`.

```r
yield_newton_hist <- Newton_Raphson_history(
  f = bond_price_error,
  df = bond_price_error_derivative,
  x0 = 0.02
)

yield_bisection_hist <- bisection_history(
  f = bond_price_error,
  a = 0,
  b = 1
)

yield_uniroot_full <- uniroot(
  f = bond_price_error,
  interval = c(0, 1)
)

yield_newton <- tail(yield_newton_hist$x1, 1)
yield_bisection <- tail(yield_bisection_hist$c, 1)
yield_uniroot <- yield_uniroot_full$root

data.frame(
  method = c("Newton-Raphson", "Bisection", "uniroot"),
```

```
  yield = c(yield_newton, yield_bisection, yield_uniroot),
  iterations = c(
    nrow(yield_newton_hist),
    nrow(yield_bisection_hist),
    yield_uniroot_full$iter
  )
)
```

```
##           method      yield iterations
## 1 Newton-Raphson 0.06759816          3
## 2      Bisection 0.06787109         11
## 3        uniroot 0.06762546          4
```

The iteration counts are not perfectly apples-to-apples because `uniroot()` uses R's built-in routine, while bisection and Newton-Raphson are our own implementations. Still, the table is useful for seeing the broad speed difference between methods.

6 Gradient Descent

Root finding solves $f(x) = 0$. Optimization solves

$$\min_x f(x).$$

Gradient descent is based on the idea that the derivative points in the direction of increase. Therefore, to decrease the function, we move in the opposite direction.

For one variable:

$$x_{n+1} = x_n - \alpha f'(x_n),$$

where α is the step size or learning rate.

6.1 Simple Quadratic Example

Let

$$f(x) = x^2 - 10x + 3.$$

The derivative is

$$f'(x) = 2x - 10.$$

The minimum occurs where $f'(x) = 0$, so the exact minimizer is $x = 5$.

```
gradient_descent <- function(df, x0, alpha = 0.2, tol = 0.0001, N.max = 100) {

  for (n in 1:N.max) {

    x1 <- x0 - alpha * df(x0)
```

```
    if (sqrt(sum((x1 - x0)^2)) < tol) {
      return(x1)
    }

    x0 <- x1
  }

  warning("Maximum number of iterations reached.")
  return(x1)
}

objective <- function(x) {
  x^2 - 10 * x + 3
}

objective_derivative <- function(x) {
  2 * x - 10
}

gradient_descent(objective_derivative, x0 = 6)
```

```
## [1] 5.000102
```

6.2 Step Size Matters

If the learning rate is too small, the method may be slow. If the learning rate is too large, the method can oscillate or diverge.

```
gradient_descent_history <- function(f, df, x0, alpha = 0.2, tol = 0.0001, N.max = 100) {

  history <- data.frame(
    iteration = 0,
    x = x0,
    f.x = f(x0)
  )

  for (n in 1:N.max) {

    x1 <- x0 - alpha * df(x0)

    history <- rbind(
      history,
      data.frame(
        iteration = n,
        x = x1,
        f.x = f(x1)
      )
    )

    if (abs(x1 - x0) < tol) {
      break
    }
```

```
    x0 <- x1
  }

  return(history)
}

hist_small <- gradient_descent_history(objective, objective_derivative, x0 = 20, alpha = 0.05)
hist_good <- gradient_descent_history(objective, objective_derivative, x0 = 20, alpha = 0.2)

head(hist_good, 10)
```

```
##    iteration         x       f.x
## 1          0 20.000000 203.00000
## 2          1 14.000000  59.00000
## 3          2 10.400000   7.16000
## 4          3  8.240000 -11.50240
## 5          4  6.944000 -18.22086
## 6          5  6.166400 -20.63951
## 7          6  5.699840 -21.51022
## 8          7  5.419904 -21.82368
## 9          8  5.251942 -21.93653
## 10         9  5.151165 -21.97715
```

The next table summarizes the final value and number of iterations for each learning rate. Notice that the large learning rate does not settle down; it keeps oscillating until the maximum number of iterations is reached.

```
gd_summary <- function(history, alpha) {
  final_row <- tail(history, 1)
  previous_x <- history$x[nrow(history) - 1]
  final_x <- final_row$x

  data.frame(
    alpha = alpha,
    final_x = final_x,
    final_f = final_row$f.x,
    iterations = max(history$iteration),
    last_step_size = abs(final_x - previous_x)
  )
}

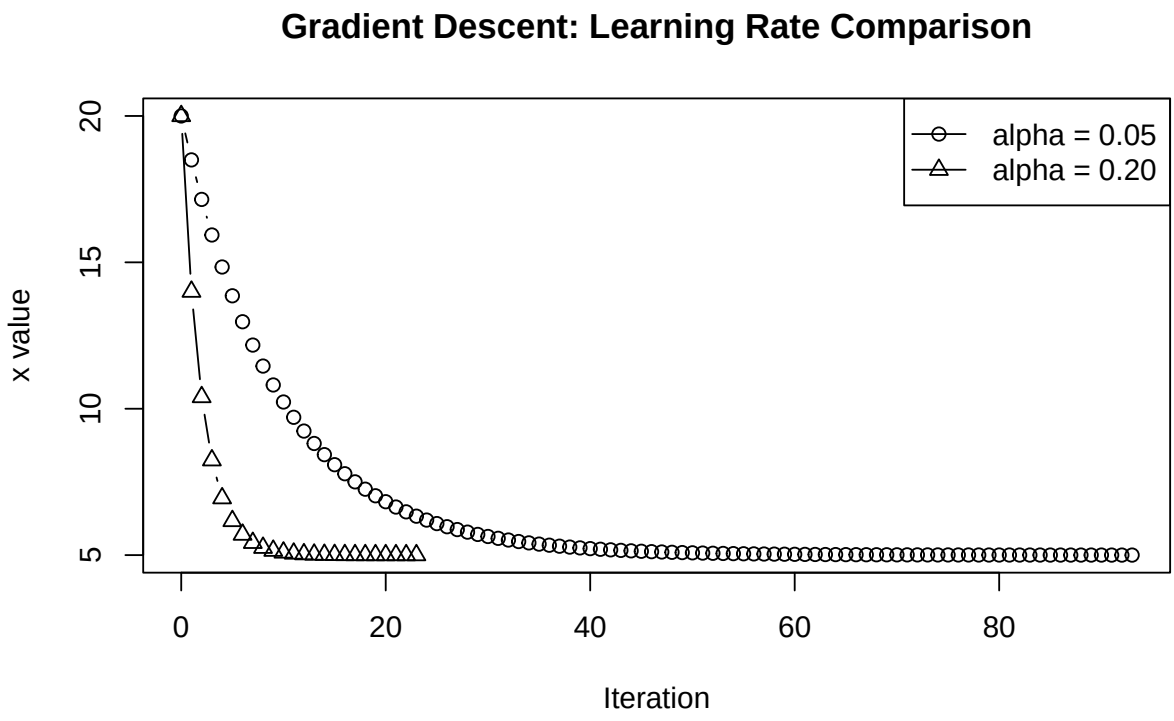
rbind(
  gd_summary(hist_small, 0.05),
  gd_summary(hist_good, 0.20)
)
```

```
##   alpha  final_x final_f iterations last_step_size
## 1  0.05 5.000833     -22         93   9.255548e-05
## 2  0.20 5.000118     -22         23   7.897302e-05
```

```r
plot(
  hist_small$iteration,
  hist_small$x,
  type = "b",
  ylim = range(c(hist_small$x, hist_good$x)),
  xlab = "Iteration",
  ylab = "x value",
  main = "Gradient Descent: Learning Rate Comparison"
)

lines(hist_good$iteration, hist_good$x, type = "b", pch = 2)

legend(
  "topright",
  legend = c("alpha = 0.05", "alpha = 0.20"),
  pch = c(1, 2, 3),
  lty = 1
)
```



7 Local Minima Example

Gradient descent can converge to a local minimum, especially when the objective function is not convex. Let

$$f(x) = x \sin(x).$$

```r
f2 <- function(x) {
  x * sin(x)
}

df2 <- function(x) {
  sin(x) + x * cos(x)
}

x0 <- 7.5
alpha <- 0.2

local_hist <- gradient_descent_history(
  f = f2,
  df = df2,
  x0 = x0,
  alpha = alpha,
  tol = 0.0001,
  N.max = 100
)

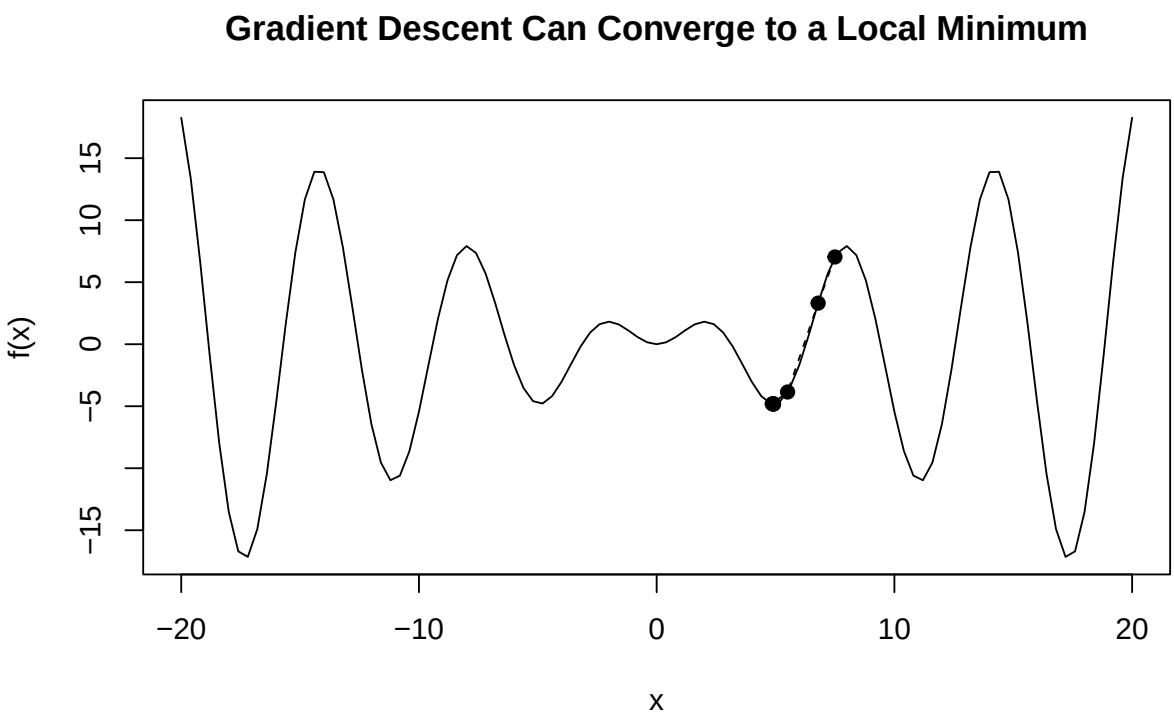
tail(local_hist)
```

```
##   iteration        x       f.x
## 2         1 6.792447  3.311541
## 3         2 5.508837 -3.852056
## 4         3 4.861059 -4.807436
## 5         4 4.914846 -4.814463
## 6         5 4.913109 -4.814470
## 7         6 4.913183 -4.814470
```

```r
curve(
  f2,
  from = -20,
  to = 20,
  xlab = "x",
  ylab = "f(x)",
  main = "Gradient Descent Can Converge to a Local Minimum"
)

points(
  local_hist$x,
  local_hist$f.x,
  pch = 19
)

lines(
  local_hist$x,
  local_hist$f.x,
  lty = 2
)
```



8 Newton's Method for Optimization

Newton-Raphson finds roots using the first derivative of a function.

Newton's method for optimization minimizes a function using both first and second derivatives.

In one dimension, the update is

$$x_{n+1} = x_n - \alpha \frac{f'(x_n)}{f''(x_n)}.$$

For the quadratic objective

$$f(x) = x^2 - 10x + 3,$$

we have

$$f'(x) = 2x - 10, \qquad f''(x) = 2.$$

```
Newton_optim <- function(df, d2f, x0, alpha = 1, tol = 0.0001, N.max = 100) {

  for (n in 1:N.max) {

    second_derivative <- d2f(x0)

    if (second_derivative == 0) {
```

```
      stop("Second derivative is zero. Newton optimization cannot continue.")
    }

    x1 <- x0 - alpha * df(x0) / second_derivative

    if (abs(x1 - x0) < tol) {
      return(x1)
    }

    x0 <- x1
  }

  warning("Maximum number of iterations reached.")
  return(x1)
}

d2_objective <- function(x) {
  2
}

Newton_optim(
  df = objective_derivative,
  d2f = d2_objective,
  x0 = 6
)
```

```
## [1] 5
```

For a quadratic function with positive second derivative, Newton's method jumps directly to the minimizer.

We can also compare Newton optimization to gradient descent using an iteration table.

```
Newton_optim_history <- function(df, d2f, x0, alpha = 1, tol = 0.0001, N.max = 100) {

  history <- data.frame(
    iteration = 0,
    x = x0,
    gradient = df(x0)
  )

  for (n in 1:N.max) {

    second_derivative <- d2f(x0)

    if (second_derivative == 0) {
      stop("Second derivative is zero. Newton optimization cannot continue.")
    }

    x1 <- x0 - alpha * df(x0) / second_derivative

    history <- rbind(
      history,
      data.frame(
```

```
        iteration = n,
        x = x1,
        gradient = df(x1)
      )
    )

    if (abs(x1 - x0) < tol) {
      break
    }

    x0 <- x1
  }

  return(history)
}

gd_optim_hist <- gradient_descent_history(
  f = objective,
  df = objective_derivative,
  x0 = 6,
  alpha = 0.2
)

newton_optim_hist <- Newton_optim_history(
  df = objective_derivative,
  d2f = d2_objective,
  x0 = 6
)

data.frame(
  method = c("Gradient Descent", "Newton Optimization"),
  approximate_minimizer = c(
    tail(gd_optim_hist$x, 1),
    tail(newton_optim_hist$x, 1)
  ),
  objective_value = c(
    objective(tail(gd_optim_hist$x, 1)),
    objective(tail(newton_optim_hist$x, 1))
  ),
  iterations = c(
    max(gd_optim_hist$iteration),
    max(newton_optim_hist$iteration)
  )
)
```

```
##               method approximate_minimizer objective_value iterations
## 1    Gradient Descent              5.000102             -22         18
## 2 Newton Optimization              5.000000             -22          2
```

9 Wrap-Up

The key ideas are:

- **Bisection** is reliable when we have a sign change on an interval.
- **Newton-Raphson** is often faster but requires a derivative and a good starting point.
- `uniroot()` is R's built-in root finding tool for one-dimensional root problems.
- **Gradient descent** minimizes a function by moving against the derivative.
- **Step size matters**: too small is slow; too large can oscillate or diverge.
- **Newton optimization** uses curvature and can be much faster when the second derivative is well-behaved.